

EXPERIMENT NO: 10

Use Ghidra to Disassemble and Analyze the Malware Code

DATE:

1. AIM

To set up a secure analysis environment, utilize the Ghidra reverse engineering framework to disassemble and decompile binary code, and interpret low-level assembly functions to identify common malware behaviors, persistence mechanisms, and indicators of compromise.

2. DESCRIPTION

Malware analysis is the critical process of dissecting compiled software to understand its capabilities, origins, and operational intent without executing it live (static analysis). Ghidra, an open-source Software Reverse Engineering framework developed by the National Security Agency (NSA), is utilized extensively by forensic investigators for this purpose. When original source code is unavailable or intentionally obfuscated by adversaries, analysts rely on Ghidra to bridge the gap between raw, machine-executable binaries and human-readable logic.

The framework's core strength lies in its CodeBrowser, which parses raw binary instructions into low-level assembly language, while its advanced Decompiler algorithmically reconstructs higher-level, C-like pseudo-code. This dual approach significantly reduces the cognitive load required to decipher complex malware algorithms. To further aid in understanding the execution logic, Ghidra generates interactive Control Flow Graphs (CFGs) that visually map out function calls, decision branches, and loops, allowing analysts to trace execution paths and identify evasion techniques.

4. PROCEDURE

Step 1: Installation, Project Initialization, and Binary Import

1. Open the Kali Linux terminal and install the Ghidra framework using the advanced packaging tool:

```
Bash: sudo apt update && sudo apt install ghidra -y
```

2. Launch Ghidra from the terminal by typing ghidra
3. create a new non-shared project titled Ghidra_Malware_Analysis.
4. Navigate to File > Import File, select the target binary (/bin/ls), and proceed with the default import configuration.

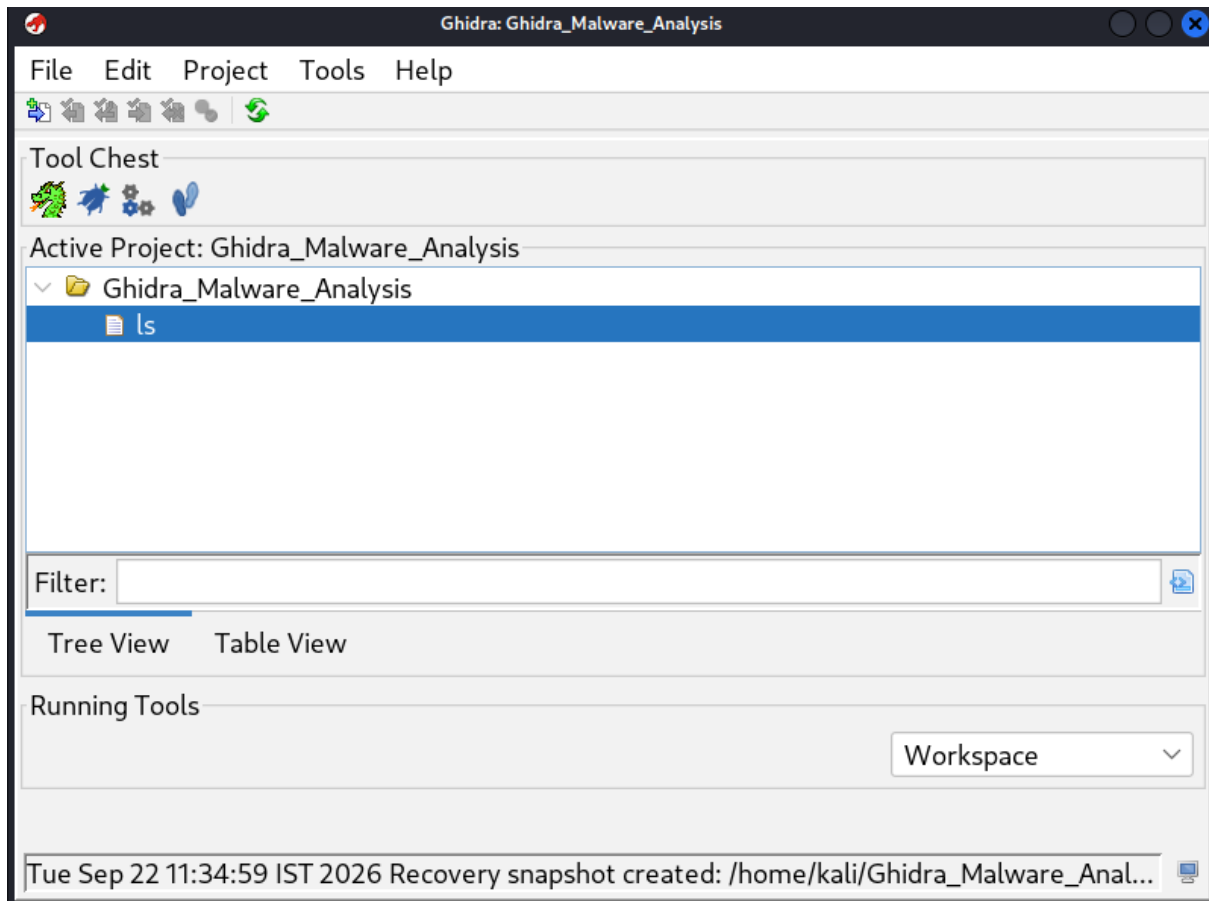


Figure 1: Initial Ghidra Project Window showing the newly imported binary

4. Double-click the imported file to open it in the CodeBrowser tool and execute the initial Auto-Analyze function when prompted.

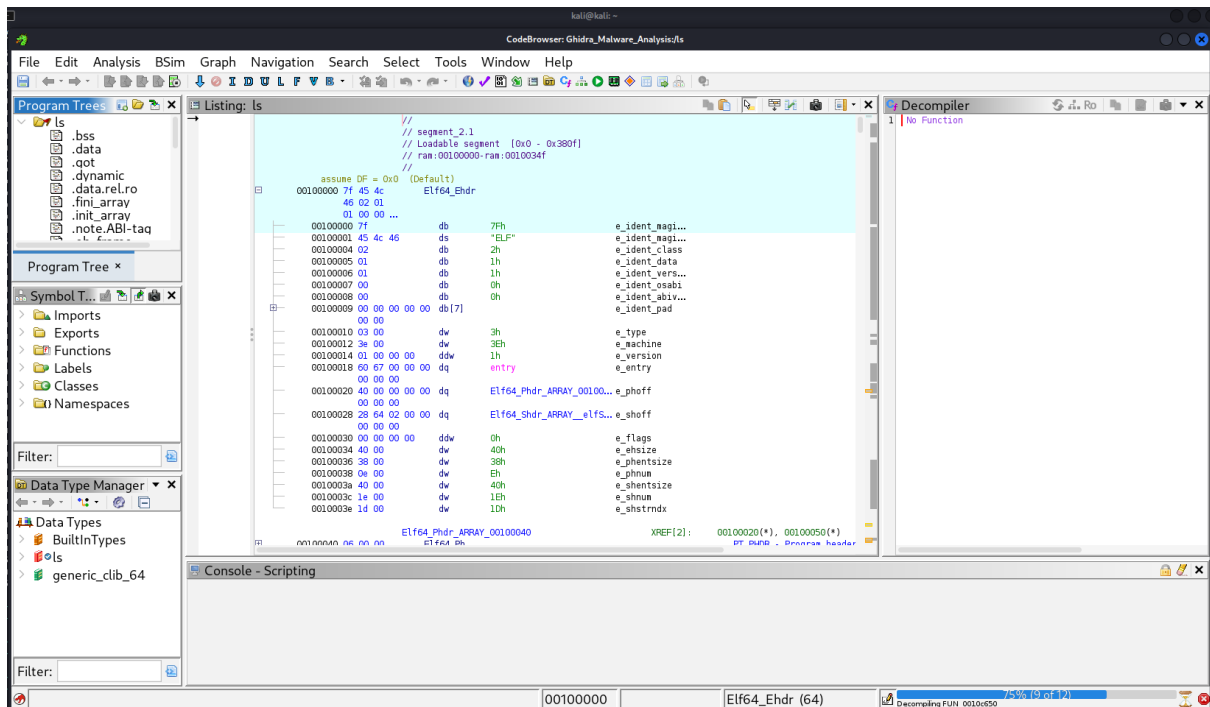


Figure 2: Ghidra CodeBrowser interface displaying the disassembly after Auto-Analysis

Step 2: String Extraction and Import Analysis

1. Navigate to Window > Defined Strings to extract hardcoded ASCII and Unicode strings from the binary.
2. In the Symbol Tree pane on the left, expand the Imports directory to identify the external shared libraries the binary relies on (e.g., libc.so.6).

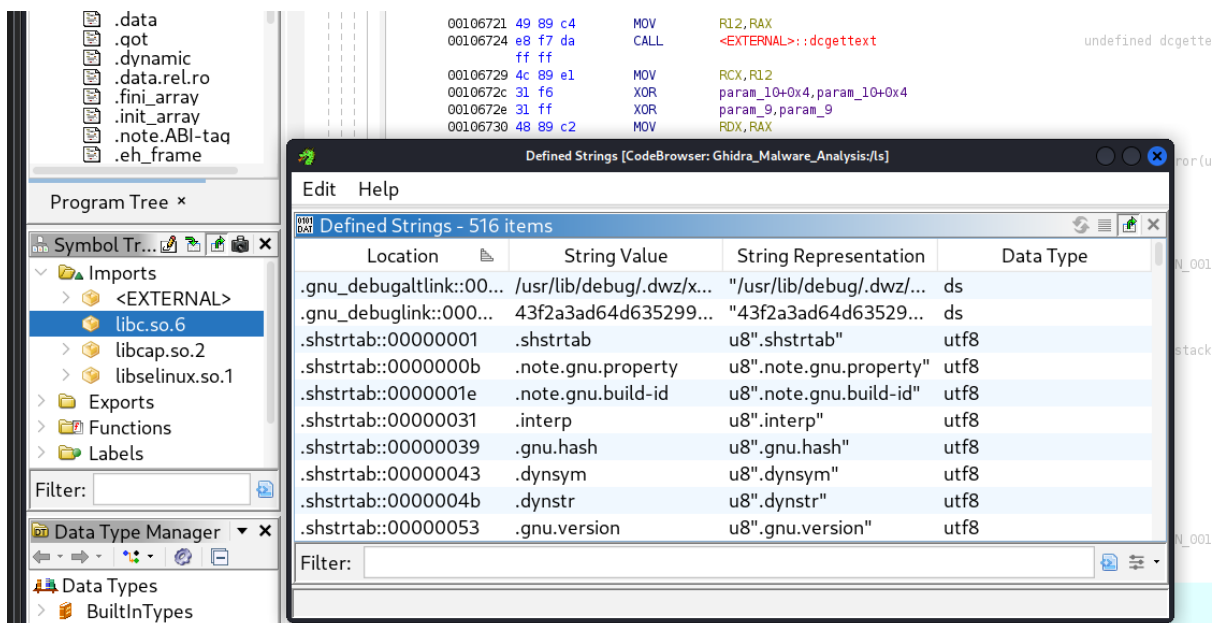


Figure 3: Extraction of Defined Strings and identification of system Imports

Step 3: Function Decomposition

1. In the Symbol Tree pane, expand the Functions directory.
2. Locate and select the entry function.
3. Observe the raw Assembly instructions in the center Listing window, and analyze the translated C-like pseudocode in the right-hand Decompile window.

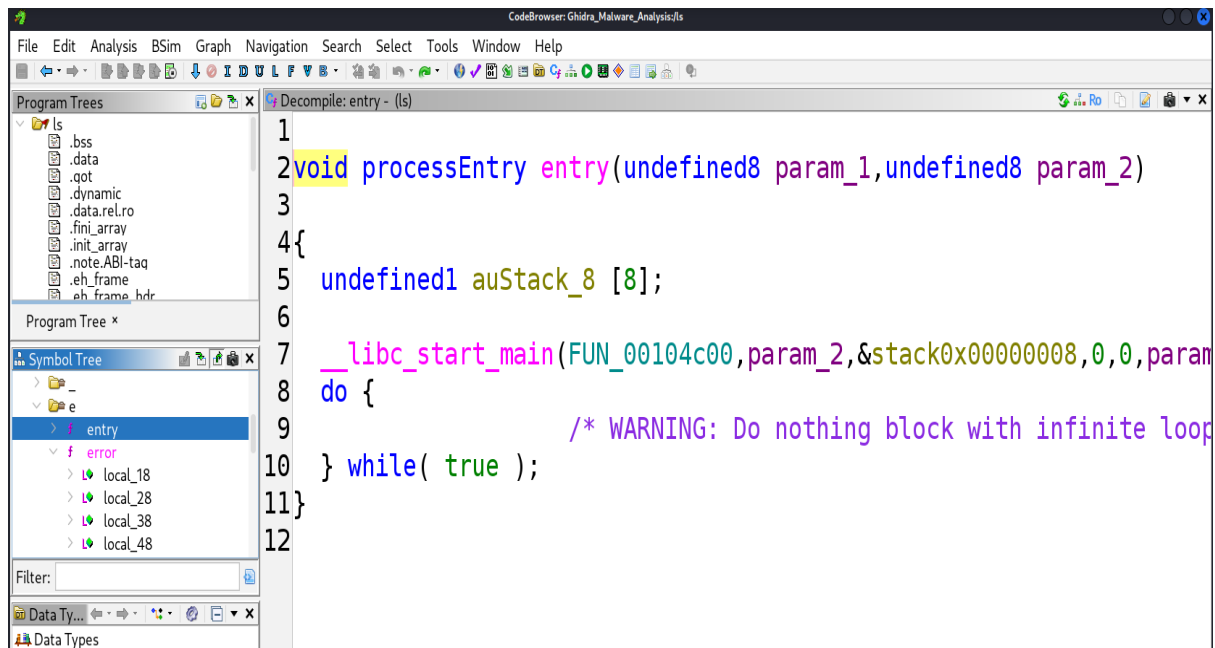


Figure 4: Decompiled C-like pseudocode of the entry function

Step 4: Execution Flow Visualization (Main Function)

1. Due to the binary being stripped, the original main function name was removed. In the Decompiler window for the entry function, identify the first parameter passed to `__libc_start_main` (e.g., `FUN_00104c00`).
2. Double-click `FUN_00104c00` to navigate to the actual main execution logic of the program.
3. With `FUN_00104c00` selected, navigate to **Window > Function Graph**.
4. Analyze the visual block diagram that illustrates the complex logical execution paths, decision branches (green and red arrows), and loops within the core code.

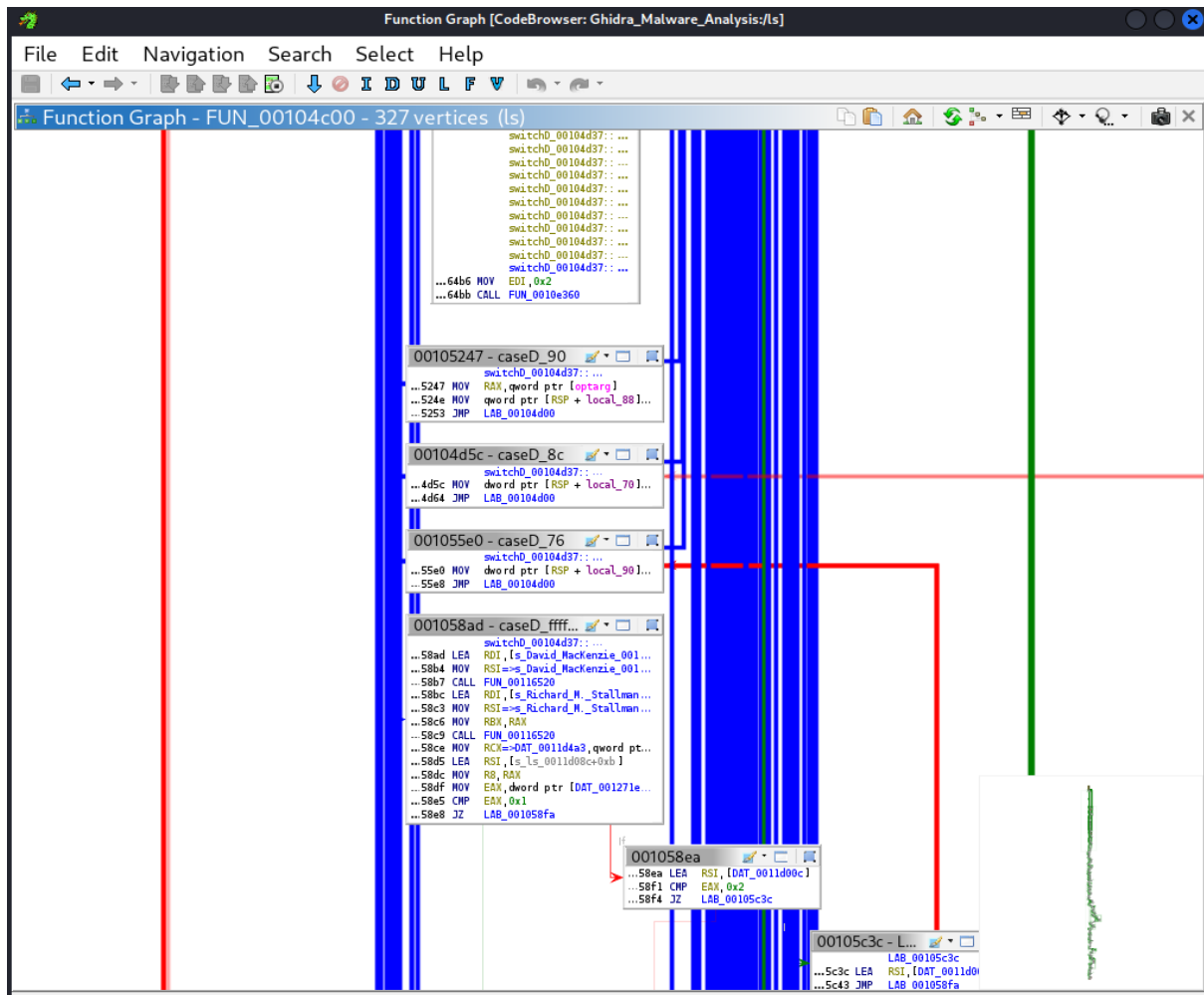


Figure 5: Visual Function Graph illustrating the complex execution flow of the stripped main function (FUN_00104c00)

5. OBSERVATIONS / FORENSIC FINDINGS

- **Code Structure:** Ghidra successfully disassembled the ELF binary, revealing its overall architecture, data sections, and metadata headers.
- **Strings & Imports:** The extracted strings and imported libraries showed that the binary depends on standard system libraries such as libc.so.6, which is typical of a Linux utility. In malware analysis, this step can help identify networking, encryption, or evasion-related dependencies.
- **Decompilation:** Ghidra's Decompiler converted the low-level assembly of the entry function into a more readable C-like representation, clearly showing the execution of `__libc_start_main`.

- **Flow Analysis:** The Function Graph provided a node-based representation of the program's execution flow, making it easier to understand the function structure, execution paths, and potential loops.
- **Stripped Binary Analysis & Symbol Resolution:** The binary had its symbol table stripped, with .symtab removed, making the original function names and variable identifiers unavailable. By examining the program's entry point and tracing the primary pointer passed to __libc_start_main, the actual main routine was identified as FUN_00104c00.

RUBRICS

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

6. RESULT

The experiment on malware analysis using Ghidra was successfully performed. The malware binary was disassembled, decompiled, and analyzed to identify its functions, strings, and behavioral patterns. This helped in understanding the internal logic, persistence mechanisms, and potential malicious activities of the code.